Prague University of Economics and Business
Faculty of Informatics and Statistics

Heuristics for Real-Time Train Dispatching

MASTER THESIS

Study program: Econometrics and Operations Research

Author: Ing. Bc. Jan Švrčina
Supervisor: Ing. Miroslav Rada, Ph.D.

Prague, May 2026

Declaration on the Use of Artificial Intelligence

1) No artificial intelligence tools were used in the preparation of the submitted work.

2) I confirm that artificial intelligence tools were used in the preparation of the submitted work. Specifically, they were used in the following ways:

- improvement of grammar and language style,
- searching for and summarizing literature,
- editing and expanding textual sections,
- generation of parts of source code,
- data generation,
- generation of images and other media,
- other (please specify).

Each use of any of the above methods is separately documented in an appendix to the thesis, which provides a more detailed explanation of which tool was used in which specific part of the work; the author's own contribution is highlighted.

I confirm that:

- my work is original and was prepared in accordance with ethical standards, in particular the Code of Ethics of the Prague University of Economics and Business; that all information sources used are properly cited in the work; and that any generated content has been verified by me;
- I accept full responsibility for the entire content of the thesis.

Acknowledgements

Poděkování.

Abstrakt

Abstrakt.

Klíčová slova

klíčové slovo, další pojem, jiný důležitý termín, a ještě jeden

Abstract

Abstract.

Keywords

keyword, important term, another topic, and another one

Contents

List of Figures

Poznámka: Seznam obrázků je vhodné použít, pokud počet obrázků v textu práce je větší než 20.

List of Tables

Poznámka: Seznam tabulek je vhodný použít, pokud počet tabulek v textu práce je větší než 20.

List of source codes

7.1 Ukázka zpracování pomocí Pythonu 42

Poznámka: Seznam výpisů programového kódu je vhodné použít, pokud počet vložených objektů tohoto typu je větší než 20.

List of abbreviations

BCC Blind Carbon Copy
CC Carbon Copy
CERT Computer Emergency Response Team
CSS Cascading Styleheets
DOI Digital Object Identifier
HTML Hypertext Markup Language
REST Representational State Transfer
SOAP Simple Object Access Protocol
URI Uniform Resource Identifier
URL Uniform Resource Locator
XML eXtended Markup Language

Poznámka: Seznam zkratek je vhodný použít, pokud počet zkratek v textu práce je větší než 20 a nejedná se o zkratky běžné. Každou zkratku je třeba při jejím prvním použití vymezit, např.: „Pro zkoumání závislosti jsme použili regresní analýzu, konkrétně zobecněné lineární modely (*generalized linear models*, GLM).“

1. Introduction

1.1 Motivation

Railway transportation remains relevant even after its long history due to its efficiency, safety, and sustainability, however the most limiting factor has always been the use of the required infrastructure. As such the critical problem to utilize the railway effectively has always been scheduling. The complexity of this problem is continuously increasing with size and intricacy of the railway system as well as the demand to use it.

Traditionally scheduling was done manually by humans, only with the advancements of optimizing algorithms and computational power are humans aided by computers. This still produces a static schedule that might be unusable if a disruption occurs. This is where train dispatching comes to play where the decisions are done in real-time, managing delays and conflicts as they happen - this part is often still only done by humans and provides a challenge for optimization algorithms as they need to find acceptable solution in the matter of minutes.

1.2 Goals

The main goal of this thesis is to develop a heuristic that is capable of train dispatching in real-time with interpretable results - a human should be able to understand the structure of the result to verify its validity.

The heuristic will be evaluated on the *DISPLIB: Train Dispatching benchmark library* (**kloster2025displibli** consisting of train dispatching problem (TDP) instances from real world data. This library of instances was introduced for an optimization competition.

A significant part of this thesis is focused on the decomposition of the problem as presented in the *DISPLIB*. This is the main innovation which allows us to use a top-down approach, reducing the problem into smaller parts and solving the conflicts in the subproblems, as opposed to the bottom-up approach, building up the solutions by resolving constrains sequentially, which is used by the winning solutions in the *DISPLIB* competition.

1.3 Thesis structure

This thesis is structured in the following chapters with their contents:

1. *Introduction* - outlines the motivation, goals and structure of this thesis.
2. *Literature review* - review of the pertinent literature separated into 3 parts. First is literature connected to the *DISPLIB*. Second are the most relevant resources inspiring

the heuristic or providing key theoretical ideas. Third is an overview of selected state of the art in scheduling optimization.

3. *Theoretical background* - basic theory necessary for the description of the solutions. Main focus is on graph topology and the use of alternative and disjunctive graphs for scheduling models.
4. *Methodology* - detailed description of the problem, decomposition and heuristic.
5. *Results* - presentation of performance of the heuristic on the *DISPLIB* instances.
6. *Conclusion* - discussion of results and resume of main ideas presented in this thesis.

2. Literature review

This chapter provides a review of the relevant literature separated into 3 parts. First is literature reverent to the *DISPLIB*. Second are the most relevant resources inspiring the heuristic or providing key theoretical ideas. Third is an overview of selected state of the art in scheduling optimization.

2.1 DISPLIB

As the main goal of this thesis is to make a heuristic that solves instances from the *DISPLIB* library, we provide a review of the introductory paper as well as a description of the competition winning solutions.

2.1.1 Problem and library

The train dispatching problem (TDP) which this thesis is working with is presented in *DISPLIB: A library of train dispatching problems* (**kloster2025displiblibrarytraindispatching**) alongside the collection of instances which can be used to benchmark this problem. This provides an important step for the subfield of train dispatching optimization - a unified problem format and real-life data to be used for benchmarking. The instances and competition results can be accessed from `https://displib.github.io/`.

> ... each study (on train scheduling) is typically tightly connected with a specific industrial use case. Code and data are seldom shared publicly. This fact hinders reproducibility, and has led to a proliferation of papers describing algorithms for more or less compatible problem definitions, without any real opportunity for readers to assess their relative performance. Inspired by the successful communities around MILP, SAT, TSP, VRP, etc., we introduce a common problem definition and file format, *DISPLIB*, which captures all the main features of train re-routing and re-scheduling. (**kloster2025displiblibrarytraindispatching**)

The instances are published with best known solutions (assumed with unlimited time), for the 10-minute computation time limit only the competition team rankings are presented, without overview of the actual solutions.

2.1.2 Competition and solutions

Part of the presentation of *DISPLIB* was also the announcement of the competition for optimization using this library. The competition concluded at the Optimization and Decision

Science Conference (ODS) 2025 where they presented the winners. Unfortunately the materials for the solutions are not available, and only one team wrote a paper about their solution. Still, we find essential to provide the reader with the description for other solutions, so the limited information from the book of abstracts for the ODS 2025 (**ods2025bookofabstracts**) is summarized.

The first place went to Carolin Scholl and Luka Stärk (DB Systel GmbH). Their solution is based on branch and bound inspired by Monte Carlo Tree Search (**DARIANO2007643**), prioritizing not only lower bound improvement but also search for feasible solutions. In each step of the tree they resolve the chronologically first conflict either by setting precedence to either of the trains on the conflicting resources or by rerouting either train if possible. This is a bottom-up approach, only adding the necessary constrains for current conflicts.

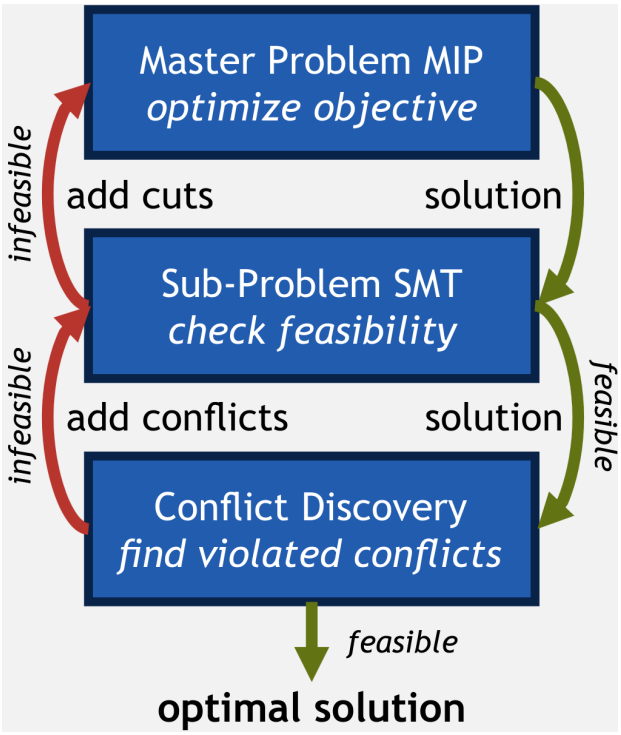
The second place went to Openbus team (ETH Zürich) which published a description of their solution in *Handling Conflicts in Microscopic Railway Timetabling at Scale: A Hybrid Approach*[1] (**Dubach2025handling**). Their solution utilizes 2 distinct solvers which run in parallel. First one is a Logic-Based Benders Decomposition (LBBD) procedure (**Leutwiler2022logic**) where a master mixed-integer programming (MIP) problem performs a selection of current available paths, slave satisfiability modulo theory (SMT) solver creates a feasible timetable if possible (without conflicts) and a subproblem detects conflicts and adds them SMT solver, which in turn adds cuts to the master problem to maintain feasibility. Figure **??** show the diagram of the solver as presented in their poster. Second solver uses a simpler MIP representation which iteratively relaxes the bounds for conflicting resources and tightens the bounds for nonconflicting resources converging to a feasible solution. Both of these solvers use a bottom-up approach, after a bound tightening procedure is applied globally. The authors argue that both of these solvers exploit different instance structures and share their bounds to improve convergence.

The third place went to Venislav Steliyanov Varbanov (independent). This solution also uses an MIP model with a decomposition to reduce the number of time variables - this idea is also used by the heuristic in this thesis, inspired by the provided description. After this aggregation of time variables, the conflicts are again resolved sequentially, similarly to the other 2 solutions.

We can see that while the winning solutions may differ in their model representation and optimization approach, and 2 of them use a certain level of aggregation, they have one thing in common - they resolve the conflicts sequentially, and thus they only need to consider the conflicts that arise in the current partial solution and therefore the number of constrains and variables necessary to prevent the conflicts stays limited.

[1] This paper is not officially published, only a pre-print was available.

Figure 2.1
Openbus DISPLIB competition poster



Source: ETH Zürich Research Collection (**Dubach2025poster**)

2.2 Theory and heuristics

While the *DISPLIB* winning solutions provided us with the ideas of aggregation of variables and combining conflict resolutions, it provided very little on the actual implementation. This section describes the relevant sources to explain the necessary theory to model the problem effectively.

Referenced by the winning solution is *A Branch and Bound Algorithm for Scheduling Trains in a Railway Network* (**DARIANO2007643**). It uses an alternative graph formulation for the train scheduling problem. With this representation we can model resource conflicts with alternative precedence arcs. It also describes the idea of inferred selection, meaning that by choosing one precedence alternative for a resource implies selection on another resources (the alternative is forbidden as it would produce an infeasible solution).

Job-shop Scheduling with Blocking and No-wait Constraints (**MASCIS2002498**) introduces the alternative graph concept as a generalization of the disjunctive graph (**roy1964problemes**). The alternative graph is used to formulate a model for the blocking job-shop problem (BJSP) that is closely related to TDP.

Improving Real-Time Train Dispatching: Models, Algorithms and Applications. (**dariano2008improving**) provides static implication rules for the alternative graph representation that can be used for

inferred selection based on the structure of TDP.

Railway scheduling problems and their decomposition (**Strotmann2007**) is thesis that show decomposition techniques that inspired the decomposition used in this thesis to split the global problem into small subgraphs using border operations and a global synchronizer to merge the solution from these subgraphs and deal with potention conflicts.

2.3 State of the art

Because the description of the solutions that are directly connected to this thesis is limited, we provide the reader with an overview of the recent state-of-the-art solutions for closely related problems in train dispatching and scheduling.

Optimizing train dispatching for the Union Pacific Railroad (**boccia2025optimizing**) presents an automated dispatching system that is in use on a real rail network. It uses a decomposition of the problem and global synchronizer to resolve conflict. It is a prime example how optimization can improve real-life solutions.

A Branch-and-Cut Algorithm for Scheduling Train Platoons in Urban Rail Networks (**chai2024branch**) describes a mixed-integer linear problem (MILP) for scheduling of train platoons, problem where different vehicles are flexibly and dynamically coupled and decoupled.

Authors in *Decentralized Multi-Agent Coordination for Real-Time Railway Traffic Management* (**damato2025decentralised**) propose the use of another emerging idea - instead of central or area based decomposition schedulers, trains cooperate as separate agents to prevent conflicts and achieve efficient scheduling. This decentralized nature of cooperation has already been used on drone related problems with success (**HertJINTHW_paper**).

3. Theoretical background

This chapter will cover the necessary theory used to formulate and solver our problem. Main focus will be on graph topology and alternative graphs and their relation to the train dispatching problem.

3.1 Basic definitions

While some of these definitions may feel redundant, we will use them as an opportunity to establish the notation. Most of the definitions are paraphrased from *Handbook of Graph Theory* (**gross2013handbook**) with notation adjusted to ours.

Graph is an ordered pair $G = (V, E)$, where $V = \{v_1, \ldots, v_{|V|}\}$ is a set of vertices[1] and $E = \{e_1, \ldots, e_{|E|}\}$ is a set of edges, where edge $e = \{u, v\}$, $u, v \in V$ is an *unordered* pair of vertices.

Directed graph (digraph) is an ordered pair $G = (V, E)$, where $V = \{v_1, \ldots, v_{|V|}\}$ is a set of vertices and $E = \{e_1, \ldots, e_{|E|}\}$ is a set of edges[2], where edge $e = (u, v)$, $u, v \in V$ is an *ordered* pair of vertices.

Multigraph is an ordered tuple $G = (V, E, r)$, where $V = \{v_1, \ldots, v_{|V|}\}$ is a set of vertices, $E = \{e_1, \ldots, e_{|E|}\}$ is a set of edges and $r : E \to \{\{u, v\} \mid u, v \in V\}$ is a mapping from edges to *unordered* pairs of endpoint vertices.

Directed multigraph is an ordered tuple $G = (V, E, s, t)$, where $V = \{v_1, \ldots, v_{|V|}\}$ is a set of vertices, $E = \{e_1, \ldots, e_{|E|}\}$ is a set of edges and $s : E \to V$ is a source (start) mapping from edges to vertices and $t : E \to V$ is a target (end) mapping from edges to vertices.

Example of each type of graph is in figure **??**.

As you can see in figure **??**, digraph allows for parallel edges (2, 3) and (3, 2). This will be a useful property for the alternative graph definition.

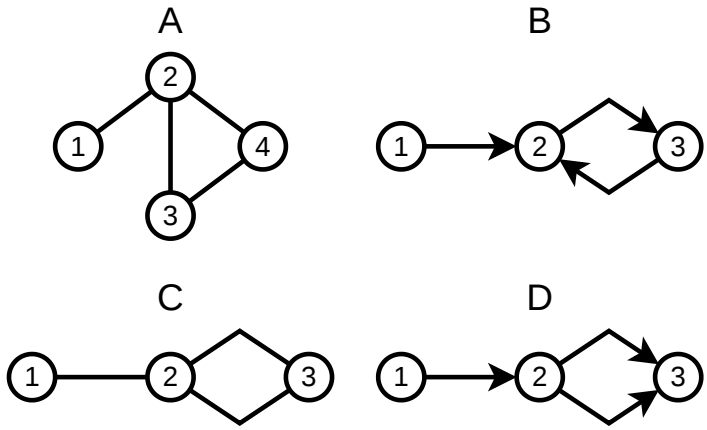
Since we will mostly be working with directed graph, to following definitions are tailored for directed graphs. The following definitions of operators also abuse the notation by omitting the specification of graph G, which will be included in the context of multiple graphs if necessary.

Source (start) $(e) = u$, $e = (u, v)$ is the starting vertex of edge e.

[1] Vertices are often called nodes.

[2] Commonly the edges in directed graphs are called arcs.

Figure 3.1
Graph types



Note: (A) is graph, (B) is directed graph, (C) is multigraph, and (D) id directed multigraph.

Target (end) $(e) = v$, $e = (u, v)$ is the ending vertex of edge e.

Incoming edges $(v) = \{e \mid t(e) = v, e \in E\}$ of vertex v is a set of edges e that end in v.

Outgoing edges $(v) = \{e \mid s(e) = v, e \in E\}$ of vertex v is a set of edges e that start in v.

Predecessors (In-neighborhood) $(v) = \{u \mid u = (e), e \in (v)\}$ of vertex v is a set of vertices u that are connected to v via its incoming edges.

Successors (Out-neighborhood) $(v) = \{w \mid w = (e), e \in (v)\}$ of vertex v is a set of vertices w that are connected to v via its outgoing edges.

In-degree $(v) = |(v)|$ is the number of incoming edges of v.

Out-degree $(v) = |(v)|$ is the number of incoming edges of v.

Path is $P = \mathrm{P}(u, v)$ from u to v is a sequence of edges $P = (e_1, \ldots, e_{|P|})$, such that $(e_{i-1}) = (e_i)$, $2 \leq i \leq |P|$, $u = (e_1)$ and $v = (e_n)$.

Cycle is a path that starts and ends in the same vertex.

3.2 Graph topology

Because event graphs described later in section **??** require certain properties or concepts such as acyclicity and topological ordering (**BangJensen2008**), this section will define and explain them.

Directed acyclic graph (DAG) is directed graph that contains no cycles.

Topological order O is a sequence of vertices in directed graph $O = (v_1, \ldots, v_{|V|})$ such that $e = (v_j, v_j) \implies v_i \prec v_j$ ($i < j$, v_i appears before v_j in the sequence).

Topological order can only be created for acyclic graphs.

Proof. By contradiction, for path starting at u and ending at v, $u \prec v$ in the topological order, because for each $e = (x, y)$ in path, $x \prec y$ and ordering $\prec$ is transitive. For cycle u and v are in the same spot in the order, this is a contradiction. □

To detect a cycle and get a topological sort we use algorithm **??**, it is a combination of depths first search (DFS) for cycle detection and DFS for topological sorting (**CLRS2022**). It runs in O($|V| + |E|$) time complexity with O($|V|$) required space. It allows us to do both cycle detection and topological sort in one pass of DFS.

Algorithm **??** uses binary operator + for concatenation of sequences, so for $S_1 = (a, b)$, $S_2 = (c, d)$, $S_1 + S_2 = (a, b, c, d)$.

[float, caption=Topological sort with cycle detection, label=alg:toposort] input: directed graph $G(V, E)$ output: () if G contains cycle else topological order O

function DFS(v, S) if label(v) = InRecursion return True else if label(v) = Done return False end

label(v) $\leftarrow$ InRecursion

foreach $w \in$(v) if DFS(w, S) return True end end

$S \leftarrow S + (v)$ label(v) $\leftarrow$ Done

return False end

begin foreach $v \in V(G)$ label(v) $\leftarrow$ NotDone end

$S \leftarrow ()$

foreach v $\in$ V(G) if DFS(v, S) return () end end

$O \leftarrow$ reverse(S) return O end

3.3 Alternative graphs

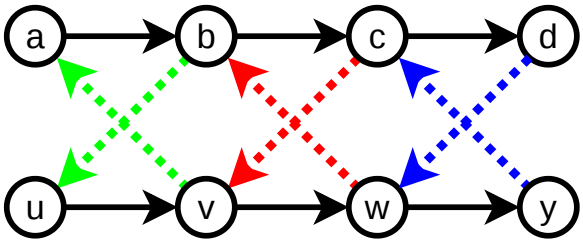
Alternative graphs were introduced in (**MASCIS2002498**). They provide a very useful framework for modeling resource allocation problems with blocking. They are a generalization of disjunctive graphs (**roy1964problemes**).

Alternative graph is an ordered tuple $G = (V, E, A)$, where $V = \{v_1, \ldots, v_{|V|}\}$ is a set of vertices, $E = \{e_1, \ldots, e_{|E|}\}$ is a set edges $e = (u, v)$, $u, v \in V$, and $A = \{a_1, \ldots, a_{|A|}\}$ is a set of disjunctive (alternative) edge pairs $a = ((u, v), (x, y))$, $u, v, x, y \in V$.

We denote the alternative $\bar{e}$ to the edge e from the disjunctive pair $a = (e, \bar{e})$.

To illustrate the concept of alternative graphs, figure **??** shows an alterative graph with 3 disjunctive pairs.

Figure 3.2
Alternative graph



Note: Alternative edges are dotted, alternative pairs grouped by color.

A selection S is a set of edges obtained by from A by choosing at most one edge from each pair. The edges in this selection are selected, their alternatives (the other edge from the pair) is forbidden.

Selection S is complete if exactly one edges is chosen from each pair, otherwise it is a partial selection.

Selection S is consistent if graph $(V, E \cup S)$ does not contain a cycle. If the graph does contain a cycle, the selection is inconsistent.

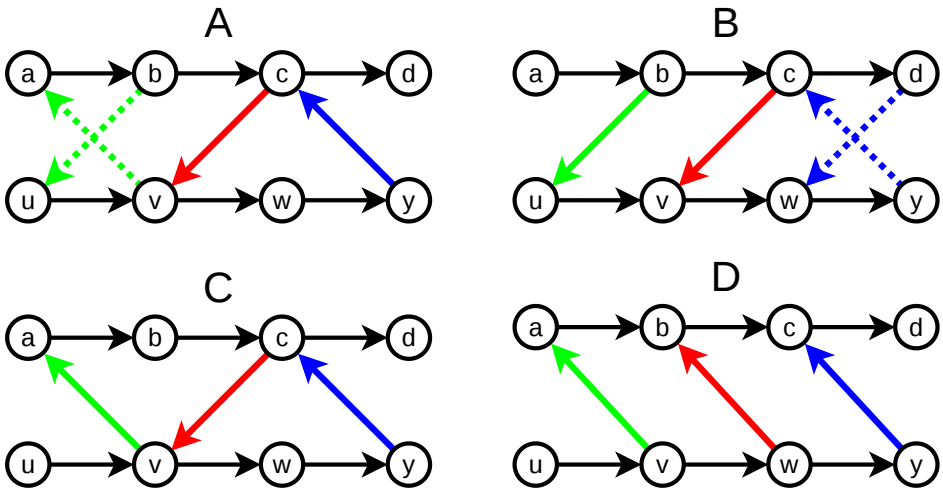
To illustrate the selection terms, figure **??** shows different types of selections from on alternative graph from figure **??**.

Alternative edge e is inferred from selection S, $S \vdash e$ if $S \cup \{\bar{e}\}$ is an inconsistent selection, meaning that the choice of e from alternative pair $(e, \bar{e})$ can be inferred from S because choosing $\bar{e}$ would be inconsistent. Similarly, we can say that one alternative edge x implies another y, $x \implies y$ if $\{x\} \vdash y$.

An example from on graphs in figure **??**, we can see that $(b, u) \implies (c, v)$ and $(v, a) \implies (w, b)$ etc.

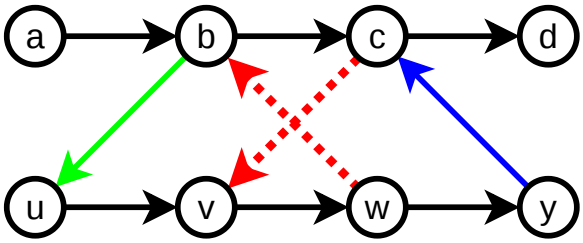
It is possible to have a partial consistent selection P for which there exist no complete consistent complete selection C, $P \subset C$. One such case is in figure **??**, where $(b, u) \implies (c, v)$ and $(x, c) \implies (w, b)$. Because (c, v) and (w, b) are in the same disjunctive pair, we cannot make a consistent selection for this pair.

Figure 3.3
Alternative graph with selections



Note: (A) is partial inconsistent, (B) is partial consistent, (C) is complete inconsistent, and (D) is complete consistent selection. Alternative edges are dotted, selected edges are full, alternative pairs grouped by color.

Figure 3.4
Partial consistent selection without possible complete consistent selection



Note: Alternative edges are dotted, selected edges are full, alternative pairs grouped by color.

3.3.1 Weighted graphs

To extend the descriptive ability of graphs to model problems we assign properties to vertices or edges. One of the most common one is to add weights to edges.

Weighted directed graph is an ordered tuple $G = (V, E, w)$, where V and E are same as in digraph definition **??**, and $w : E \to \mathbb{R}$ is an edge weight mapping.

Length $\mathrm{L}(\cdot)$ of path P is the sum of the weights for each edge,

$$\mathrm{L}(P) = \sum_{e \in P} w_e \tag{3.1}$$

Length of the longest path $\mathrm{L}_{\max}(\cdot)$ from u to v is

$$\mathrm{L}_{\max}(u, v) = \max_{P \in \mathcal{P}_{u,v}} \mathrm{L}(P), \tag{3.2}$$

where $\mathcal{P}_{u,v}$ is the set of all paths from u to v.

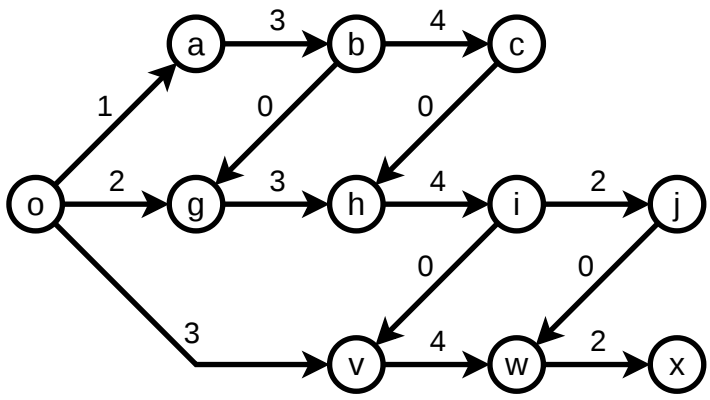
The length of the longest path can only be calculated for a graph without any positive length cycles, otherwise we could simply go in the cycle indefinitely to prolong the path (**gross2013handbook**).

Event graphs

In our model, we will use weighted DAG we call an *event graph*, where each vertex corresponds to an event, edge direction assigns precedence of events and the weight of an edge will be corresponding to a minimal duration between events. All events share the origin o which determines the start of the timeline, and it is used to calculate the time of each following event.

Example Figure **??** shows an event graph for 3 trains sharing resources. Edges (o, a), (o, g), and (o, u) set the earliest start of the train, horizontal edges define the fastest passing of the section and diagonal edges ensure precedence for resources e.g. edge (b, g) ensures that g starts after b, since section (a, b) required the same resource as (g, h).

Figure 3.5
Event graph for 3 trains with shared resources



The time of the event x in the even graph can be calculated as the longest path from origin o to x, $\mathrm{T}(x) = \mathrm{L}_{\max}(o, x)$.

Finding all the paths from o to each $x \in V$ would be very expensive, by DFS the complexity is exponential (**homer2011computability**).

First optimization is the use of Bellman principle (**light2024principle**)(**bellman1952dynamic**),

$$\mathrm{T}(v) = \mathrm{L}_{\max}(o,\, v) = \max_{u \in (v)} \mathrm{L}_{\max}(o,\, u) + w_{(u,\, v)}. \tag{3.3}$$

This is principle is used by the Floyd-Warshall algorithm (**floyd1962algo**) and Dijkstra's algorithm (**Dijkstra1959**) which are used for finding the shortest paths for all pairs for vertices and from a select starting vertex respectively. Both of them could be adjusted to find the longest path within polynomial complexity, $\mathrm{O}(|V|^3)$ for Floyd-Warshall and $\mathrm{O}((|V| + |E|)log|V|)$ for Dijkstra with binary heap.

Since we already use algorithm **??** to detect a cycle, we can just process the time of the events in the topological order, because topological order guarantees that any predecessors are already processed.

Critical path $\mathrm{CP}(\cdot)$ for vertex v is the longest path from origin o to v.

Critical path is limiting the earliest start of event v, delaying any event along this path will also delay the event v,

$$\mathrm{L}(\mathrm{CP}(v)) = \mathrm{T}(v). \tag{3.4}$$

Critical path edge $\mathrm{CPE}(\cdot)$ of v is the last edge of the critical path for v.

We can again use the Bellman principle to reconstruct the critical path,

$$\mathrm{CP}(v) = \mathrm{CP}(u) \cup \{(u,\, v)\},\ \mathrm{CPE}(v) = (u,\, v). \tag{3.5}$$

Algorithm **??** outputs the time $\mathrm{T}(\cdot)$ and critical path edge $\mathrm{CPE}(\cdot)$ for each vertex in the event graph. It runs in $\mathrm{O}(|V| + |E|)$ time complexity.

In our event graph, the first vertex in the topological order will always be the origin o as it is the only vertex v with $(v) = 0$.

[float, caption=Time and critical path predecessor calculation, label=alg:time] input: weighted graph G, topological order O output: time T and critical path edge CPE for each vertex in G begin foreach $v \in$ V(G) T(v) $\leftarrow$ 0 CPE(v) $\leftarrow$ Undefined end

foreach u $\in$ O $\setminus$ o foreach (u, v) $\in$(u) if T(v) $<$ T(u) + $\mathrm{w}_{u,v}$ T(v) $\leftarrow$ T(u) + $\mathrm{w}_{u,v}$ CPE(v) $\leftarrow$ (u, v) end end end end

Example For event graph in figure **??**, the time and critical path edge output from algorithm **??** is in table **??**. So the vertices in the critical path for a are (o, a, b, c), for j are (o, a, b, c, h, i, j) and for w are (o, a, b, c, h, i, u, v, w).

Table 3.1

Time and critical path edge for vertices in event graph

vertex	T	CPE
a	1	(o, a)
b	4	(a, b)
c	8	(b, c)
g	4	(b, g)
h	8	(c, h)
i	12	(h, i)
j	14	(i, j)
u	12	(i, u)
v	16	(u, v)
w	18	(v, w)

4. Methodology

4.1 Problem description

Most of the problem description is taken from the *DISPLIB format specification* (**kloster2025displibspec**), a concise explanation is in accompanied paper (**kloster2025displiblibrarytraindispatching**).

Each instance in the *DISPLIB* library consist of set of trains and an objective to be minimized. All data is presented in JSON format (**pezoa2016foundations**). Train is defined by an operation graph which is DAG, operation has the following content:

- minimal duration
- start lower bound (optional)
- start upper bound (optional)
- list of successors
- list of required resources
  - resource name
  - release time

Data **??** show and example of an operation entry in the instance JSON.

All operation graphs have one entry (start) operation, with no predecessors, and one exit (end) operation, with no successors. Each operation ends exactly when the successor operation starts (this is what we call a transition), exit operation has no end.

Resources are anything that is required by the operation, typically physical sections of the railway network. Resources can only be used by one operation at the time and are released after the end of the operation that uses it plus the defined release time.

Objective is defined by a list of objectives components, each with the following content

- train
- operation
- threshold
- coefficient (linear objective)
- increment (stepwise objective)

While the format allows both coefficient and increment to be non-zero, in practice none of the instances mix both in one objective component.

Data **??** show and example of an objective component entry in the instance JSON.

The total objective J is defined as a sum of components for active operations only,

$$J = \sum_{o \in O_{a,J}} c_o \max\{0, t_o - \bar{t}_o\} + d_o H(t_o - \bar{t}_o), \tag{4.1}$$

where $O_{a,J}$ is a set of active operations that have an objective component, t_o is start of the operation, $\bar{t}_o$ is the threshold for the operation, c_o is the linear objective coefficient, d_o is the stepwise objective increment, and $H(\cdot)$ is Heaviside step function (**abramowitz1972handbook**).

A solution to the problem is a sequence of events, each event entry corresponds to start of an operation and contains:

- train
- operation
- time

Solution is also presented in a JSON format, data **??** shows truncated solution example. To reiterate, the end event of an operation is the start event of its successor.

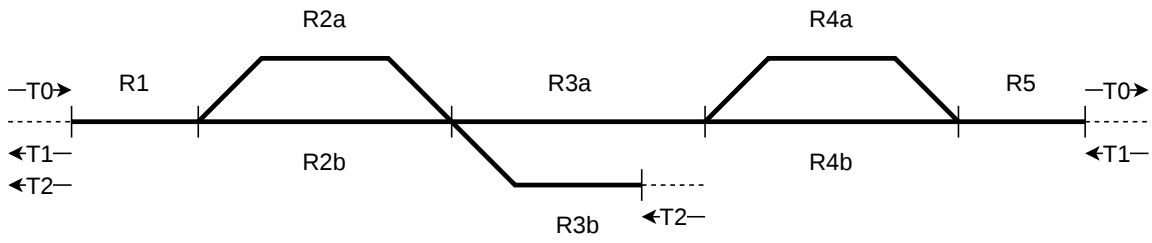
> - The sequence of events is given in chronological order, i.e., the time of each event is greater than or equal to the time of all preceding events.
> - The subsequence of operations applying to each train forms a path through that train's operations graph, starting at the entry operation and ending at the exit operation.
> - The time value of each start event satisfies the lower and upper bounds on the start time of the operation.
> - The duration from each start event to the corresponding end event must be greater than or equal to the minimum duration of the corresponding operation.
> - For any pair of operations o_1 and o_2, where:
> - the start event of o_1 precedes the start event of o_2 in the global sequence, and o_1 and o_2 belong to different trains, and
> - o_1 and o_2 use a common resource r,
>
> the following must hold:
> - the end event for o_1 precedes the start event for o_2 in the global sequence, and
> - the duration from the end event of o_1 to the start event of o_2 is greater than or equal to the release time of resource r in o_1.
>
> (**kloster2025displibspec**)

While the events in the solution have a time, their order in the list is still significant, as they define in which order the resources are unlocked and locked. This is to prevent a resource swap situation described in section **??**.

4.1.1 Example problem

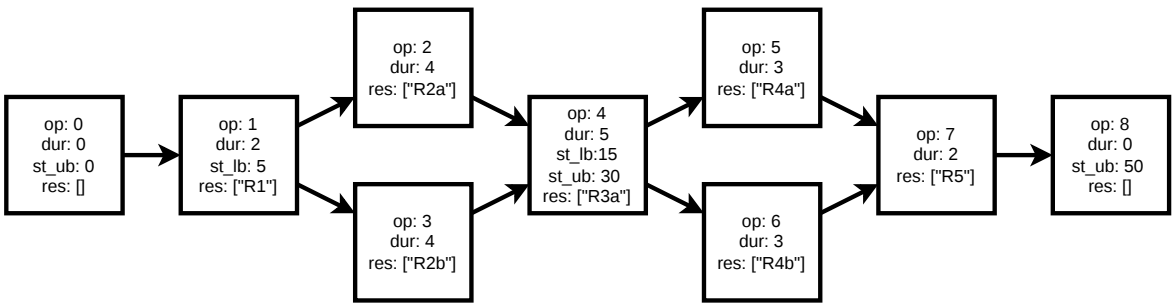
To illustrate the format we have prepared example problem, the railway network diagram is in picture **??**. The network consists of 3 trains, 18 operations, and 8 railway sections, corresponding to resources.

Figure 4.1
Example railway network



The corresponding operation graph for train 0 (first) is in figure **??**, JSON entry for operation 4 is in data **??**, and JSON entry for objective component for the last operation is in data **??**.

Figure 4.2
Operation graph for train 0



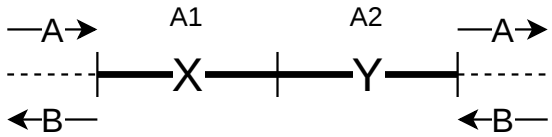
[float=htbp!, caption=JSON operation entry, label=data:op] $"\text{start}_l b" : 0, "start_u b" : 50, "min_d uration" : 5, "resources" : ["resource" : "R3a", "release_t ime" : 0], "successors" : [5, 6]$ [float=htbp!, caption=JSON objective component entry, label=data:obj] $"\text{type}": "\text{op}_d elay", "train" : 0, "operation" : 8, "threshold" : 25, "coefficient" : 1$

[float=htbp!, caption=JSON solution truncated, label=data:events] "events": [..., "train": 0, "operation": 5, "time": 15 , "train": 1, "operation": 4, "time": 15 , ...], $"\text{objective}_v alue" : 25$

4.1.2 Resource swap

Consider the situation as in figure **??**, where 2 trains a and B are passing a section of 2 resources X and Y in opposite direction. If we were to set events A2 and B2 to the same time, we would not have a resource collision on either resources (overlap of intervals for operations A1 and B2 on resource X or A2 and B1 on resources Y) but in reality the trains would collide.

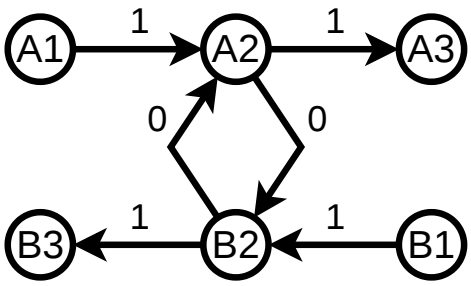
Figure 4.3
Resource swap railway



In this situation, resource Y is being acquired by a while holding X and X is being required by B while holding Y.

We can detect the resource swaps using an event graph by cycle detection, like in figure **??** where A2 and B2 form a cycle.

Figure 4.4
Resource swap event graph



Using cycles in event graph to detect resource swaps only works if all resource precedence edges are present, otherwise we still might have a resource swap situation because the current event times do not create a collision and one or both of the edges that would create the cycle in the event graph are missing.

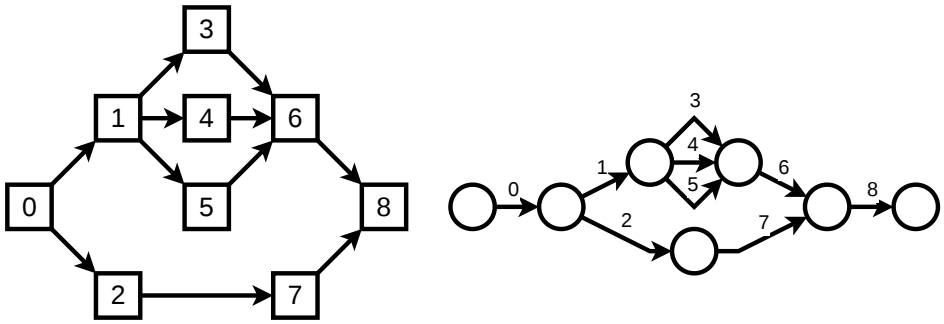
4.2 Problem aggregation

To effectively model the problem we will aggregate the operations and resources from the initial problem into several interconnected structures.

4.2.1 Junctions

First aggregation is what we call junctions, corresponding to a single event of one operation starting and one ending (except for starting and ending operations). Because we always want each operation to have single determined event for start and end, the junction graph is a directed multigraph, where each edge corresponds to an operation (as opposed to an operation graph where each operation was a vertex), figure **??** shows and example of an operation graph with corresponding junction graph. We use the name junctions as conceptually, the operations converge and diverge in these points as do real tracks in railway junctions.

Figure 4.5
Operation graph and junction graph



Note: Operation graph is left, junction graph is right.

Aside from more natural representation of using vertices as single events and edges as intervals, which translates to the events graphs, we also reduce the number of required vertices and edges, for the example in figure **??**, it is from 9 vertices and 11 edges to 7 vertices and 9 edges, that is including the often redundant start and end events.

Disjoint set

To aggregate the variables into junctions, we use a *disjoint set* (**GallerFischer1964**). It is often called the *union-find* data structure, it used to partition elements into non-overlapping (disjoint) sets. It has 2 basic operations:

- *Find:* Unary operation, returns to what set item belongs to, usually by returning the representative element.

- *Union:* Binary operation, if the 2 elements belong to different sets, it joins them together.

In short, each element has a parent entry, which is at the start initialized to itself and the representative element of a set always has the parent as itself. To get the representative we traverse the tree upwards via parent entries until we find the representative. When we use the union operation on 2 elements from different sets, the representative of one becomes a representative of the other (if they belong to the same set nothing happens).

To optimize the data structure we use path compression (flattening of the parent tree by assigned the parent of the element to their parent when traversing the tree) and union by size (always attaching smaller tree to bigger tree) (**Tarjan1975**). Both operations with optimizations are in algorithms **??** and **??**.

[float, caption=Disjoint set - find, label=alg:disjset$_{f}ind]input : element voutput : representatives$

begin while $u \neq \text{parent}(u)$ $\text{parent}(u) \leftarrow \text{parent}(\text{parent}(u))$ $u \leftarrow \text{parent}(u)$ end $s \leftarrow u$ return s end

[float, caption=Disjoint set - union, label=alg:disjset$_{u}nion]input : elements uand voutput : none$

begin $u \leftarrow \text{find}(u)$ $v \leftarrow \text{find}(v)$ if $u \neq v$ if $\text{size}(u) < \text{size}(v)$ $\text{parent}(u) \leftarrow v$ $\text{size}(v) \leftarrow \text{size}(v) + \text{size}(u)$ else $\text{parent}(v) \leftarrow u$ $\text{size}(u) \leftarrow \text{size}(u) + \text{size}(v)$ end end end

Procedure

The procedure to create the junctions is done for each train separately.

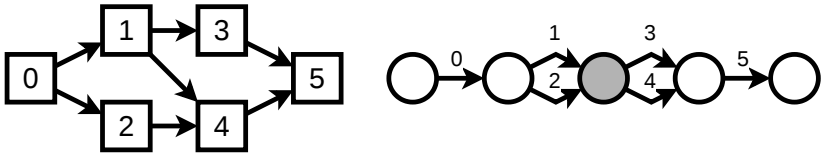
1. Create a disjoint set for each operation of the train.
2. For each operation o we use the *union* operation on each consecutive pair of successors $\{x\, y\}$, x, $y \in \text{succ}(o)$.
3. Each disjoint set of operations is a set of outgoing operations (j) for junction j.
4. Add the end (sink) junction j_{end} that represents the end of the exit operation o_{exit} for the train, it has no outgoing operations.
5. Assign the source junction $\text{s}_{\text{junct}}(o) \leftarrow j$ for each operation $o \in (j)$.
6. Assign the target junction $\text{t}_{\text{junct}}(o) \leftarrow \text{s}_{\text{junct}}(x)$ to any of its successors $x \in \text{succ}(o)$ (they all must have the same starting junction because of step 2.).
7. Assign the target junction $\text{t}_{\text{junct}}(o_{\text{exit}}) \leftarrow j_{\text{end}}$ to the end (sink) junction of the train.
8. Assign each junction the set of incoming operation $(j) \leftarrow \{o \mid \text{t}_{\text{junct}}(o) = j\}$.

Invalid transitions

We could have a possible situation where the set of outgoing operations for a junction j is not equal to the set of successors for an incoming operation o_{in}, $(j) \neq \text{succ}(o_{\text{in}})$, or vice versa the set of incoming operations for a junction j is not equal to the set of predecessors for an outgoing operation o_{out}, $(j) \neq \text{pred}(o_{\text{in}})$. This generates an invalid transition $(o_{\text{in}}, o_{\text{out}})$, such that $o_{\text{in}} \in (j)$ and $o_{\text{out}} \in (j)$, but $o_{\text{out}} \notin \text{succ}(o_{\text{in}})$ or $o_{\text{in}} \notin \text{pred}(o_{\text{out}})$. This case is rare and only affects a few instances, but should be checked when making routing decisions.

An example of such invalid transition is in figure **??**, where the transition (2, 3) in the third junction is invalid.

Figure 4.6
Operation graph junction graph for invalid transition

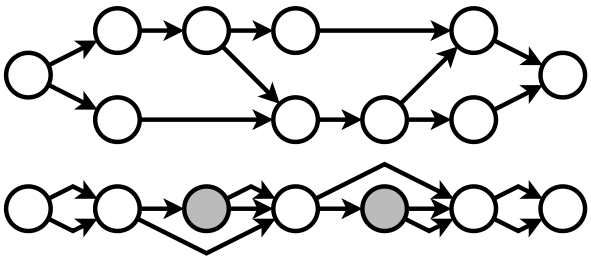


Note: Operation graph is left, junction graph is right. Junction with invalid transition (2, 3) is gray.

4.2.2 Levels

Second is the aggregation to levels. We group parallel junctions to levels,[1] as junctions can be parallel. This is useful to reduce the number of required vertices for each train in the event graph. An example of a junction graph and its corresponding level graph is in figure **??**.

Figure 4.7
Junction graph and its corresponding level graph



Note: Junction graph is at the top, level graph is at the bottom. Optional levels are gray.

[1] In (**Dubach2025handling**) they call levels *event clusters*, the level term was taken from Venislav Varbanov.

Each level L consists of set of junctions $j \in L$ that are parallel and thus mutually exclusive. The incoming and outgoing set of operations for each level is the union of the incoming and outgoing operations of its junctions, respectively,

$$(L) = \bigcup_{j \in L} (j), \tag{4.2}$$

$$(L) = \bigcup_{j \in L} (j). \tag{4.3}$$

The start (source) level contains only the start (source) junction, and the end (sink) level contains only the end (sink) junction.

Required and optional levels

Not all levels need to be passed when traversing the graph. Such levels we call optional, the levels that have to be passed when traversing the graph we call required. In (**Dubach2025handling**) they use the concept of required levels to generate bound constraints. We use required levels to identify border levels, which are used to create routing sections in **??**. In the example from figure **??** the grayed out levels are optional, because they can be bypassed by an operation.

Topological generations

To aggregate the junctions into levels, we use the concept of *topological generations* (**Hagberg:2008:Networ**
To create these generations we use adjusted Khan's algorithm for topological sorting (**Kahn:1962:Topologic**
where we group all junctions (vertices) currently without incoming operations (edges) into a level as shown in algorithm **??**.

[float, caption=Topological generation algorithm, label=alg:topogen] input: junction graph $G(J, O)$, junctions J (vertices), operations O (edges) output: levels $(L_0, \ldots, L_n)$ (subsets of junctions)

begin foreach $j \in J$ indeg$(j) \leftarrow (j)$ end

$k \leftarrow 0$ $Z \leftarrow \{J_0\}$

while $Z \neq \emptyset$ $L_k \leftarrow Z$ $Z \leftarrow \{\}$ foreach $i \in L_k$ foreach (i, j) in (i) indeg$(j) \leftarrow$ indeg$(j) - 1$ if indeg$(j) = 0$ $Z \leftarrow Z \cup \{j\}$ end end end $k \leftarrow k + 1$ end

return $(L_0, \ldots, L_n)$ end

Procedure

We apply algorithm **??** to each train separately. We also assign each operation to each level based on the junctions and assign each operation its starting and ending level.

To get required levels, we first label all levels as required and then iterate over all operations and check which level it bypasses (if any), and mark those as optional.

4.2.3 Routes

Now that we have aggregated distinct operation start and end events into junctions and levels, we aggregate the routing decision into routes. Each route R consists of set of operations such that if operations x and y are in the same route and y is a successor of x then, x is the *only* predecessor of y and y is the *only* successor of x, and to reduce the number of routes we force the implication the other way as well,

$$((x \in R) \wedge (y \in R) \wedge (y \in \mathrm{succ}(x))) \iff ((\mathrm{succ}(x) = \{y\}) \wedge (\mathrm{pred}(y) = \{x\})). \tag{4.4}$$

This follows the train flow network definition in (**fuchs2025optimizing**). An example of how operations are grouped into routes is in figure **??**.

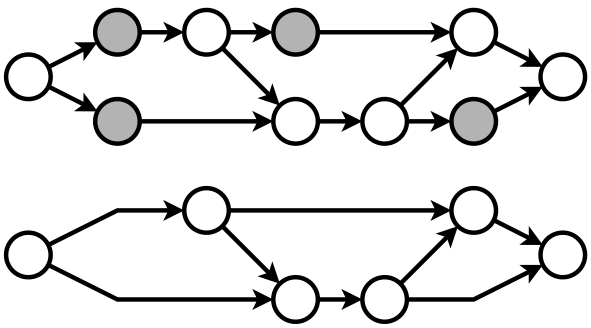
If operation y is a successor of x then x is a predecessor of y,

$$y \in \mathrm{succ}(x) \iff x \in \mathrm{pred}(y). \tag{4.5}$$

Activation z_R of route R is equal to activation of its operations z_o,

$$z_R = z_o, \forall o \in R. \tag{4.6}$$

Figure 4.8
Junction graph and route graph



Note: Junction graph is at the top and route graph is at the bottom. Non-routing junctions are gray.

This assignment of routes also generates a set of routing junctions (junction is routing if any route ends and/or begins there), each having a routing (flow) constraint

$$\sum_{R \in \rho_{\text{in}}(j)} z_R - \sum_{R \in \rho_{\text{out}}(j)} z_R = \begin{cases} 1 & \text{if } j \text{ is a start (source) junction} \\ -1 & \text{if } j \text{ is an end (sink) junction} \\ 0 & \text{otherwise} \end{cases}, \tag{4.7}$$

where z_R is the activation of R, $\rho_{\text{in}}(j)$ is a set of incoming routes and $\rho_{\text{in}}(j)$ is a set of outgoing routes for junction j.

For non-routing junctions, the constraint is redundant as it has only one incoming and one outgoing route which are the same.

Procedure

For each train we iterate over operations in topological order and assign them the same route if it is the only successor of its only predecessor, otherwise we create a new route. Procedure follows algorithm **??**.

[float=htbp!, caption=Route assignement, label=alg:route] input: operations $O = (o_0, \ldots o_n)$ in topological order output: route assignment $R : O \to \mathbb{Z}_0^+$

begin $k \leftarrow 0$

foreach $o \in O$ if $\{p\} = \text{pred}(o) \wedge \{o\} = \text{succ}(p)$ $R(o) \leftarrow R(p)$ else $R(o) \leftarrow k$ $k \leftarrow k + 1$ end end

return r end

4.2.4 Routing sections

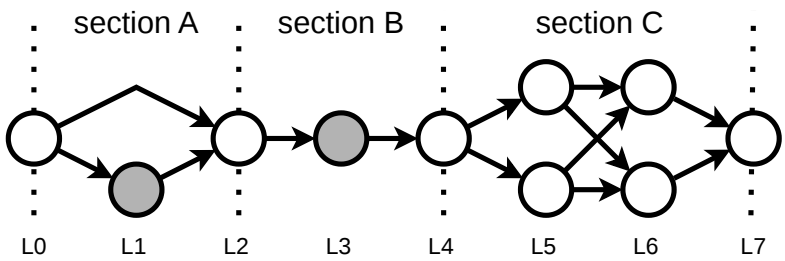
To separate what routing decisions are connected to each other and which are independent, we now group routes into sections. Each section starts and ends at a border level. Border levels are identified using conditions in the following subsection. Example of a junction graph being split into sections is in figure **??**.

4.2.5 Border levels

Border level is a level that meets the following conditions:

(a) The level is required.
(b) All junctions $j \in L$ for level L are routing junctions.

Figure 4.9
Junction graph with identified sections and levels



Note: L0, L2, L4, and L7 are border levels. Non-routing junctions are gray.

(c) For all incoming operation $o_{\text{in}} \in (L)$ of level L, the successor set of o_{in} is equal to the outgoing operation set of L, $\text{succ}(o_{\text{in}}) = (L)$.
(d) For all outgoing operation $o_{\text{out}} \in (L)$ of level L, the predecessor set of o_{out} is equal to incoming operation set of L, $\text{pred}(o_{\text{out}}) = (L)$.

Condition (a) ensures that each section starts and ends at a required level. Condition (b) ensures that we do not split a route by the border level. Conditions (c) and (d) ensures that any route from the level can be taken independently of which route was taken to the level and, vice versa, any route to the level can be taken independently of which route was taken from the level.

A sufficient set of conditions to replace conditions (c) and (d) is:

- The level has one junction.
- The single junction has no invalid transitions.

This is due to the nature of how we create the junctions in the first place. If a level L has a single junction j, then $(L) = (j)$ and $(L) = (j)$, and there are no invalid transitions in this junction, then the following equations hold,

$$(L) = (j) = \text{pred}(o),\ \forall o \in (L), \tag{4.8}$$

$$(L) = (j) = \text{succ}(o),\ \forall o \in (L). \tag{4.9}$$

Start and end levels

The start level (source) and end level (sink) are always border levels,

(a) The start level (source) and end level (sink) are always required.
(b) The start level (source) and end level (sink) have always one junction, the start (source) and end (sink) junction, respectively.
(c) There are no incoming operations for the start (source). The end (sink) has one incoming operation, the exit, which has no successor.

(d) There are no outgoing operations for the end (source). The start (source) has one outgoing operation, the entry, which has no predecessor.

Procedure

For each train we do the following:

1. Identify the border levels with the aforementioned conditions.
2. Assign consecutive sections ($S_0, \ldots, S_n$) based on border levels.
3. Add all routes R within border levels (using its operations' levels) to the specified section $S \leftarrow S \cup \{R\}$.

Example

In example from figure **??** we can see that L1 satisfies conditions (c), and (d) for a border level, but it is not required, and its junction is non-routing, so it does not satisfy condition (a) and (b).

We can also see that L3 satisfies conditions (a), (c), and (d), but again, its junction is non-routing, so (b) is not satisfied.

Levels L5 and L6 satisfy (a) and (b), but (c) and (d) are not satisfied. This is because the top and bottom outgoing operations of L4 have different disjunct successor sets thus different ending junctions, and, respectively, the top and bottom incoming operations of L7 have disjunct predecessor sets and thus different junctions as well.

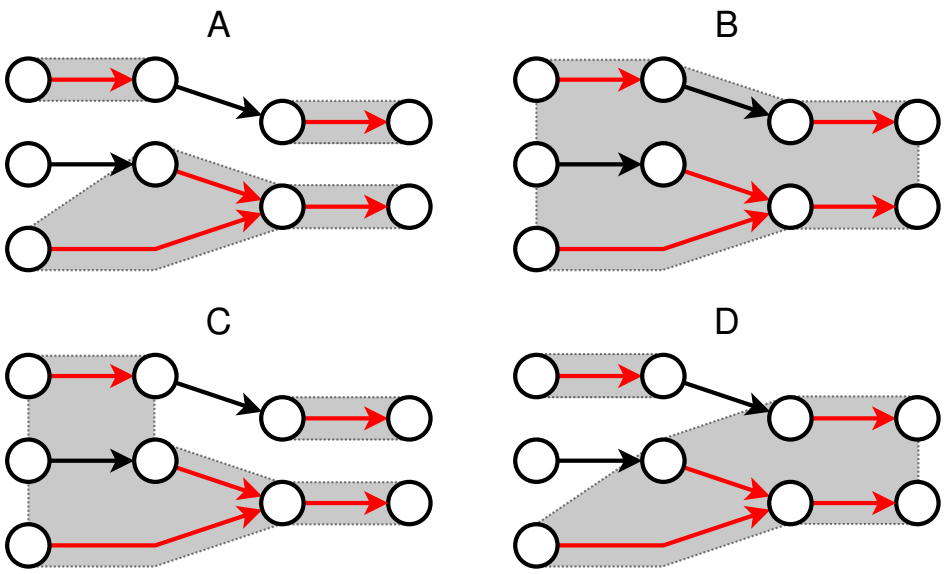
4.2.6 Resource chunks

So far we have only done aggregation based on the operations and their successors. Now we do aggregation based on the resources. While this is inspired by (**Dubach2025handling**), the implementation may differ as the authors did not provide us with sufficient details, only the conditions that are used to create the chunks.

The goal is to merge the resource uses of the same resource for directly connected operations (by predecessor and successor sets) as well as merge parallel uses of the resource. What we do not want to group is reentries of resource, where on train acquires a resource, then releases it, and then acquires it again. Figure **??** shows an example of how operations using the same resource are merged.

The (B) case in figure **??** shows an invalid chunk, where the operations on the top have a reentry of the same resource, thus they cannot be in the same chunk.

Figure 4.10
Resource chunks



Note: Operations using the selected resource are red. (A) are chunks after direct merges, (B) is an invalid chunk with reentry, (C) and (D) are valid chunks after parallel merges.

While direct merges might feel sufficient, in the following section **??**, we will be creating a data structure that grows with the amount of chunks, so reducing the number of possible chunks will be helpful to reduce the requirements of said structure.

Each chunk $c = (r_c, O_c)$ is defined by the source r_c and its set of operations O_c. We say that the chunk is active if any of the operations $o \in O_c$ are active. Each operation o also has its set of chunk C_o, one chunk for each resource it requires.

Chunk c is active $z_c = 1$ if and only if any of the operations $o \in O_c$ are active,

$$z_c = 1 \iff \exists o \in O_c,\ z_o = 1. \tag{4.10}$$

The start of chunk is the start of its first active operation (this holds for start junction, level, and time). The end of the chunk is the end of its last active operation (this again holds for start junction, and level, end time is the end time of the last operation plus the release time for the resource).

Procedure

We apply the following procedure for each train separately and for each resource that is used by the train:

1. Get a set of all operation for a train that are using the selected resource r.
2. Create a *disjoint set* data structure like in section **??** for each operation using resource r.
3. For each operation union the sets of each predecessor and each successor with the operation of they share the same resource. These are the direct merges.
4. Iterate over the pairs of disjunct sets $\{O_a,\ O_b\}$ and check if any of the operations $y \in O_b$ are reachable from any operation in $x \in O_a$, and vice versa check if any of the operations $x \in O_a$ are reachable from any operation $y \in O_b$. If they are not reachable, union O_a and O_b. These are the parallel merges. Note: This is an iterative procedure, the following checks to other chunks are done for the whole merged set of operations, $O_c = O_a \cup O_b$.
5. Assign each chunk $c = (r, O_c)$ its set of operations O_c and add c to the set of chunks C_o for each operation $o \in C_o$.

To check the reachability we use a simple breath first search (**moore1959shortest**) from the starting set of operations that has a cutoff based on the maximum level of the target operations to reduce the search space.

4.3 Link graph

This section describes a data structure that is used to generate implication rules for resource precedence of chunks. This will be useful to reduce the number of required variables and constrains in the following model for conflict resolution. We will also use it to explore how routing decision change the implication rules and how these decision may cause unwanted effects for scheduling.

While the implication rules for resource precedence are discussed in (**MASCIS2002498**), and (**dariano2008improving**) shows the structural implication rules for train scheduling, the only relation to routing is in (**Dubach2025handling**) where they only use a specific set of conditions.

We will try to broaden the set of conditions that can be used to generate the structural implication by searching pairs of matching pairs of chunk conflicts using this data structure. Because we will be maintaining a set of links between chunks, we are calling it *link graph*.

4.3.1 Resource chunk links

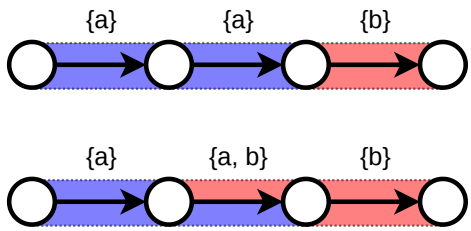
Because of the blocking nature of resource use, where we need to acquire all the resources r_b, $b \in C_y$ for the chunks C_y of operation y to transition from operation x to y, and only then can we release the resources r_a, $a \in C_x$, we say that the chunks C_x are *linked* to chunks C_y, so each chunk $a \in C_x$ has a forward link $f = (a,\ b)$ to each chunk $b \in C_y$, and each chunk $b \in C_y$ has a backward link $g = (b,\ a)$ to each chunk $a \in C_x$.

Each operation o can have multiple chunks, so we create both forward $f = (a, b)$ and backward $g = (b, a)$ for each pair of chunks (a, b), $\forall a,\ b \in C_o,\ a \neq b$, creating what we call a direct link as opposed to the aforementioned transition link.

We keep track of forward links F_c and backward links G_c for each chunk c separately, so we can have $(c, a) \in F_c$ and also $(c, a) \in G_c$ at the same time, which is the case for direct links on chunks on the same operation o, where $c, a \in C_o$.

It is important to note that the link is active if any of the transitions (x, y) that create it are active (both x and y are active), or it is direct link via the operation o that is active. Figure **??** shows both a transition and a direct link example.

Figure 4.11
Resource chunk link



Note: Operations have their chunk sets. Chunk a is blue, chunk b is red. Transition link is at the top, direct link is at the bottom.

Conflicts and precedence

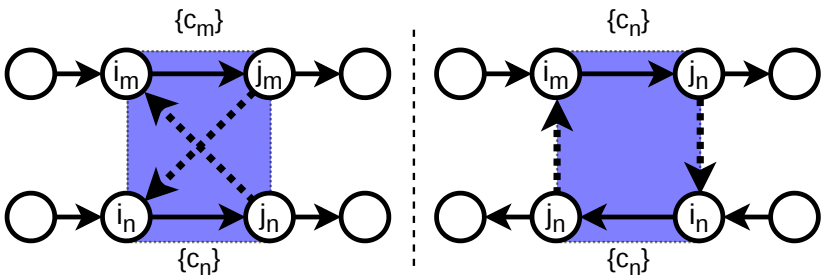
A conflict $k = \{c_m, c_n\}$ is a pair of chunks c_n and c_m on the same resource $r = r_{c_m} = r_{c_m}$ and for different trains $m = \text{train}(c_m) < \text{train}(c_n) = n$. We resolve each conflict by assigning precedence $p_k = 1 - \bar{p}_k$, meaning that if $p_k = 1$, the resource r will be released by c_m before it is acquired by c_n, if $\bar{p}_k = 1$, the resource r will be released by c_n before it is acquired by c_m.

We can model this situation using alternative graph where each vertex is an event (junction or level), each edge is an operation and the alternative pair is the precedence decision. Figure **??** show the alternative graph for a conflict. By assign precedence $p_k = 1$, we select edge $(j_1,\ i_2)$ from the alternative pair $((j,\ k),\ (l,\ i))$ and vice versa.

Without the information about the resources before and after the conflict, we cannot determine if the trains pass the resource in the same or opposite direction. This can be seen from **??** where both alternative graphs are equivalent.

Figure 4.12

Alternative graph for conflict



Note: Alternative edges are dotted. Conflict chunks are in curly braces.

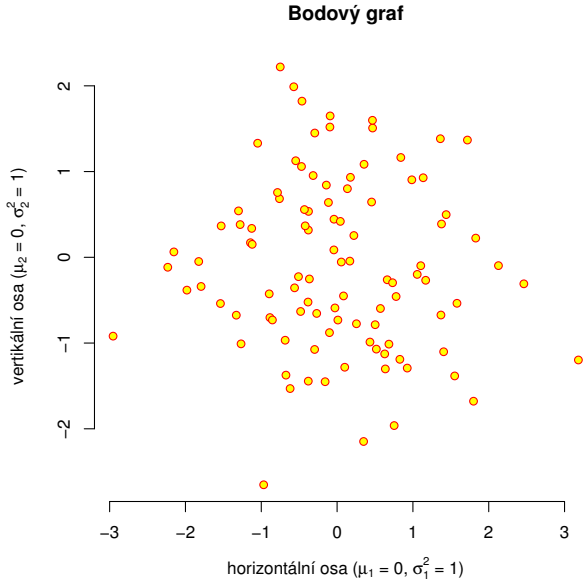
5. Results

6. Conclusion

7. Dummy

Figure 7.1

Náhodný výběr z rozdělení $\mathcal{N}_2(\mathbf{0}, I)$



Poznámka: Vlastní zpracování na základě dat z ČSÚ.

Table 7.1

Maximálně věrohodné odhady modelů 1 a 2

	1	**2**
Abs. člen	−10,01***	42,01**
	(1,01)	(1,89)
Pohlaví (muž)	9,89*	8,16
	(5,98)	(8,18)
Výška (cm)	0,78***	(0,12)

Poznámka: (i) V závorkách jsou uvedeny směrodatné chyby získané na základě 500 iterací neparametrického bootstrapu.
(ii) * p < 0,05, ** p < 0,01, *** p < 0,001.
(iii) Vlastní zpracování na základě dat z ČSÚ.

Source code 7.1

Ukázka zpracování pomocí Pythonu

```
import numpy as np

def incmatrix(genl1,genl2):
    m = len(genl1)
```

```
    n = len(genl2)
    M = None #to become the incidence matrix
    VT = np.zeros((n*m,1), int)  #dummy variable

    #compute the bitwise xor matrix
    M1 = bitxormatrix(genl1)
    M2 = np.triu(bitxormatrix(genl2),1)

    for i in range(m-1):
        for j in range(i+1, m):
            [r,c] = np.where(M2 == M1[i,j])
            for k in range(len(r)):
                VT[(i)*n + r[k]] = 1;
                VT[(i)*n + c[k]] = 1;
                VT[(j)*n + r[k]] = 1;
                VT[(j)*n + c[k]] = 1;

                if M is None:
                    M = np.copy(VT)
                else:
                    M = np.concatenate((M, VT), 1)

                VT = np.zeros((n*m,1), int)

    return M
```

Appendices

A. Název první přílohy

B. Název druhé přílohy